

How Zcash Hides a Transaction

and what the zero-knowledge proofs actually prove

m@rek.onl

What a transparent ledger publishes

- A transparent ledger (Bitcoin, Zcash's transparent pool) records, for every output, *its spending conditions and its amount*; spending publicly links it to the outputs of another transaction.
- Identifying the people behind addresses needs further information.
- That is enough raw material for address clustering, amount tracing, and the full transaction graph.

A shielded payment must achieve the same effects — transfer of value, prevention of double-spending, conservation of supply — without disclosing the individual notes, their recipients, or their hidden amounts.

Four requirements, four mechanisms

(the talk's roadmap)

1. Represent value hidden yet provable → notes + note commitments
2. Prove possession of a committed note without identifying which commitment tree → the
3. Prevent double-spending without identifying the spent notes → nullifiers
4. Prove no value created without revealing amounts → value commitments + binding signature

One zero-knowledge proof per bundle binds the per-Action facts together. Signatures, public balance checks, and the node's nullifier set complete the guarantees.

The data model

A note

$$\mathbf{n} := (\text{addr}, v, \rho, \psi, \text{rcm})$$

- Address $\text{addr} = (d, \text{pk}_d)$ — an 88-bit diversifier and a transmission key; value $v \in \{0, \dots, 2^{64} - 1\}$ zatoshi.
- The uniqueness seed $\rho \in \mathbb{F}_{p_{\text{Pallas}}}$ is public: it is the paired spend's nullifier. The secret $\psi \in \mathbb{F}_{p_{\text{Pallas}}}$ helps mask the eventual nullifier; $\text{rcm} \in \mathbb{F}_{p_{\text{Vesta}}}$ is the commitment trapdoor.
- All three are bound into the note commitment.

The note itself never touches the chain.

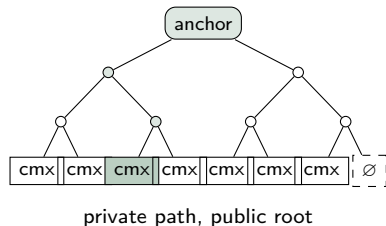
The chain stores a sealed envelope

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(\text{g}_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi)) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$$

- The commitment field is $\text{cm}_x := \text{Extract}(\text{cm})$, the x-coordinate; ciphertexts and other Action fields are also public.
- With an independent uniform blinder, the commitment is *perfectly hiding*. Actual seed-derived blinders and ciphertexts require their own security assumptions.
- *Computationally binding*: reopening one commitment as two notes yields a discrete-log solver under the binding reduction. No nontrivial commitment scheme is both perfectly hiding and perfectly binding.
- The hash is Sinsemilla, chosen for *in-circuit* cost: one table lookup per 10-bit chunk — because the spender must recompute cm inside the proof.

One tree hides them all

- Each pool appends every cmx in creation order to its own Merkle tree of fixed depth 32.
- Each block's final root is an *anchor*.
- The private path hides the leaf among those under that anchor; external information may narrow this set.
- Anchors have no age limit; their blocks must remain on the main chain. Nullifiers prevent double-spends.
- Recent anchors offer larger trees; unusual choices can fingerprint wallets.



The nullifier: a one-time serial number

$$\text{nf} := \text{Extract}([F_{\text{nk}}(\rho) + \psi] G^{\text{nf}} + \text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}}$$

with $F_{\text{nk}}(\rho) = \text{PoseidonHash}(\text{nk}, \rho)$ and G^{nf} a nothing-up-my-sleeve generator.

- Two requirements that pull in opposite directions:
- *Deterministic for a given note* — every ingredient is fixed once the note exists, so spending twice yields the same nf twice, and the network rejects the second.
- *Unlinkable without the nullifier key* — under the Poseidon PRF assumption, or DDH with the output-range condition stated in the Orchard Book. A full viewing key can recognise spends.
- Chained uniqueness: each new note is created with $\rho_{\text{new}} := \text{nf}_{\text{old}}$, so nullifier uniqueness on chain forces ρ -uniqueness within that pool (Faerie resistance).
- The nullifier key nk is deliberately not the spend key: viewing can be delegated without spending.

The mental model: spend = prove membership + reveal serial

$$\mathcal{R}_{\text{mem}} = \left\{ \left(\underbrace{\text{rt}}_{\text{public}}; \underbrace{(\nu, i, \pi)}_{\text{private}} \right) : \text{Verify}(\text{rt}, i, \nu, \pi) = \text{accept} \right\}$$

1. Locate your leaf, its path, its position in the public tree.
2. Choose a recent anchor that contains it.
3. Prove in zero knowledge: membership under the anchor, note well-formedness, correct nullifier derivation.
4. Publish the proof and the nullifier — *but neither the leaf, nor its position, nor the path.*

The root must be public — everyone verifies against agreed state.

The path must be private — nobody links the spend to a leaf.

Anatomy of an Action

Every Action spends *one* old note and creates *one* new note — a deliberately fixed shape that conceals whether it is “really” a spend, an output, or both.

Hidden (witness)

- the old note and the new note
- keys, Merkle path + position
- randomisers α , rcv

Public (per Action)

(anchor, cv^{net} , nf_{old} , rk , cmx)

plus the ciphertexts (epk , C_{enc} , C_{out})

Per *bundle*: one anchor shared by every Action, one aggregate Halo 2 proof over all Actions, one SpendAuthSig per Action, one declared net flow v_{balance} + one binding signature.

What the proof actually proves

The Action statement

The SNARK proves, with **public inputs**

(anchor, cv^{net} , nf_{old} , rk, cmx, enableSpend, enableOutputs, disableCrossAddress)

and the note, keys, Merkle path, randomiser α , and trapdoor rcv as **private witness**, the conjunction A1–A10.

- These circuit inputs occupy *ten field elements*; the remaining circuit data are witness. (Points enter as coordinate pairs.) Other transaction fields, including ciphertexts and signatures, are public too.
- Checks A1–A9 are the NU5 statement; A10 arrives with NU6.3 (*Ironwood*).
- The next six slides summarise the checks. The protocol and *Ironwood Guide*, “The Action statement”, also specify canonical encodings, non-identity requirements, and exceptional cases.

A1/A2 — both notes really are notes

Old note opens the commitment used by the membership check:

$$\text{NoteCommit}_{\text{rcm}_{\text{old}}}(\mathbf{g}_d^* \parallel \mathbf{pk}_d^* \parallel \text{LE}_{64}(v_{\text{old}}) \parallel \text{LE}_{255}(\rho_{\text{old}}) \parallel \text{LE}_{255}(\psi_{\text{old}})) \in \{\text{cm}_{\text{old}}, \perp\}$$

New note, with $\rho_{\text{new}} := \text{nf}_{\text{old}}$:

$$\text{Extract}(\text{NoteCommit}_{\text{rcm}_{\text{new}}}(\dots)) \in \{\text{cmx}, \perp\}$$

(Both checks allow a $\perp$ escape for Sinsemilla's incomplete-addition exceptions — a disjunction costing only a negligible soundness term.)

- **Prevents:** opening one leaf as two different notes (binding = DL), and publishing a cmx that opens to nothing.
- The forced chaining $\rho_{\text{new}} := \text{nf}_{\text{old}}$ makes output ρ values unique within each pool — the chain already enforces nullifier uniqueness there.

A3 — the spent note is a real leaf

Recompute the root from $\text{Extract}(\text{cm}_{\text{old}})$ up the witnessed path of length 32; then

$$v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$$

- **Prevents:** spending positive value from a note nobody ever created.
- The gate by v_{old} permits a *zero-value dummy* spend to skip membership — how an Action pads with no real input.
- **The escape hatch is arithmetic, not a flag:** value zero *is* the permission to skip, and a zero-value note mints nothing.
- Each tree layer is hashed with its height tag, so a path at one height cannot replay at another.

A5 — the published serial number is the right one

$$\text{nf}_{\text{old}} = \text{Extract}([F_{\text{nk}}(\rho_{\text{old}}) + \psi_{\text{old}}] G^{\text{nf}} + \text{cm}_{\text{old}})$$

- **Prevents:** emitting a fresh random “nullifier” per spend — which would allow unlimited double-spending.
- A1, A5, and A7 bind the note and its nullifier key, so a second spend yields the same tag and the network rejects it.
- Faerie resistance for distinct notes relies on discrete log on Pallas; it does not require collision resistance of Poseidon.

The double-spend *lookup* is not here: whether nf was already seen concerns global history. This circuit delegates the lookup to the node; other protocols can prove non-membership against a public set commitment.

A6/A7 — one identity signs, views, and nullifies

Spend authority: the published signature key is a fresh blinding of the owner's,

$$rk = ak + [\alpha] G^{\text{SpendAuth}}$$

Address integrity: the keys, the address, and the nullifier key belong to one identity,

$$\text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk) \in \{ivk, \perp\}, \quad pk_d = [ivk] g_d$$

(The $\perp$ -weakened form again, as in A1/A2.)

- With SpendAuthSig, **Prevents**: spending someone else's note; it also prevents substituting an unrelated nk to obtain new nullifiers.
- The same ak that A6 re-randomises is welded, through a binding commitment, to the nk of A5 and to the address of the spent note.
- Independent uniform α per Action makes rk *statistically* unlinkable across spends.

A4 — the value delta is committed, in range

$$cv^{\text{net}} = [v_{\text{old}} - v_{\text{new}}] V^{\text{Orchard}} + [rcv] R^{\text{Orchard}}, \quad v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$$

with V, R independent nothing-up-my-sleeve base points.

- **Prevents:** committing to anything but the true net change — and *overflow minting*: the sums to come live in a 255-bit field, so unconstrained values could wrap subtraction into free money. The 64-bit range checks close that.
- Each Action publishes one Pedersen envelope holding its value delta; nobody can read it, *everyone can add them* — the balance story, two slides ahead.

A8/A9/A10 — consensus gates

$$v_{\text{old}} \cdot (1 - \text{enableSpend}) = 0 \quad v_{\text{new}} \cdot (1 - \text{enableOutputs}) = 0$$

- **Prevents:** smuggling real value through a context that forbids it — a coinbase must set $\text{enableSpend} = 0$, so only zero-value dummies pass.
- From NU6.3, four more product gates of the same shape ($\delta := \text{disableCrossAddress}$) force the new note's receiver to equal the spent note's when $\delta = 1$, as full curve points — sealing the legacy Orchard pool to self-transfers and turnstile exits.

Statement map: A1–A2 represent notes, A3 proves membership, A5 (with the chain's nullifier set) prevents double-spending, A4 conserves value, A6–A7 bind the authorising key to the note (SpendAuthSig supplies authority); A8–A10 are consensus-level gating.

What the statement does *not* include

- **Double-spend freshness** — the nullifier-set lookup concerns global history; the node does it.
- **Anchor validity** — the proof ties root to leaf; the node checks the root was ever a main-chain anchor.
- **Balance across Actions, fees, transparent flows** — pure public-key arithmetic, next slide.
- **Authority at spend time** — the SpendAuthSig verifies outside the circuit, under rk, over the transaction digest.
- **The ciphertexts** — entirely unproven; the recipient's own trial-decryption checks close forgery there.

The circuit proves per-Action consistency; signatures, balance checks, and state lookups complete transaction validation.

Balance without amounts

A public key only a balanced transaction can know

Envelopes add. Consensus value and transaction-size bounds prevent modular wraparound. Summing the Action commitments in a bundle:

$$\sum_{i=1}^m cv^{\text{net},i} = \left[\sum_i (v_{\text{old},i} - v_{\text{new},i}) \right] V^{\text{Orchard}} + [\text{bsk}] R^{\text{Orchard}}, \quad \text{bsk} := \sum_i rcv_i$$

The network subtracts the *declared* net flow:

$$bvk := \sum_i cv^{\text{net},i} - [v_{\text{balance}}] V^{\text{Orchard}} \quad \text{balanced} \iff bvk = [\text{bsk}] R^{\text{Orchard}}$$

- If the books balance, the V -component cancels and bvk is a key whose secret the author knows; if not, knowing its discrete log would break independence of V and R .
- Balance is proved by *signing* with it: the 64-byte binding signature is a Schnorr proof of knowledge of bsk .
- bvk is not independently chosen: the commitments determine it, and the verifier recomputes it from public data.

The signature layer: one scheme, two meanings

Both signatures are RedPallas — verify $[s]P = R + [c]X$ with $c = H^*(R \parallel X \parallel \text{sighash})$ — differing only in the base point and in what knowing the secret *means*:

	base P	key X	knowing the secret means
SpendAuthSig (per Action)	$G^{\text{SpendAuth}}$	rk	authority over the spent note
binding sig (per bundle)	R^{Orchard}	bvk	the accounts balance

- Re-randomisation: $\text{rsk} := \text{ask} + \alpha$, $\text{rk} = [\text{rsk}] G^{\text{SpendAuth}}$ — for uniform α the published key is uniform *independently* of ak : the *key's* unlinkability is information-theoretic (the full spend also needs the proof and signature to leak nothing, and α to stay secret). The circuit (A6/A7) vouches the shift started from the note owner's key.
- Both sign the ZIP-244 sighash — a digest covering every Action, the flags, the balance, the anchor, and the network-upgrade ID (ZIP-244 for v5; ZIP-229 extends the digest to v6 and Ironwood). This binds the signed fields and prevents replay across different consensus branch IDs.

Three layers of trust

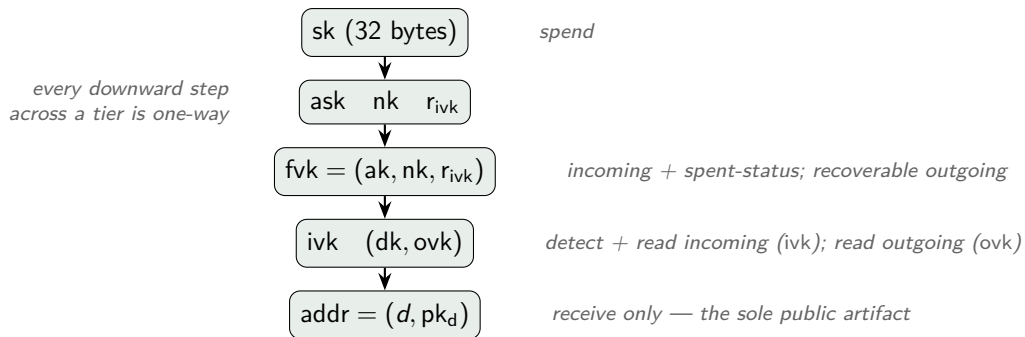
Halo 2 circuit	homomorphic sum	RedPallas signatures
per-Action facts A1–A10 cannot see across Actions	hidden deltas = declared flow cannot tell deltas are well- formed	authority + balance-knowledge cannot trace rk to an owner

Illustrative ordinary shielded checks (not all transaction consensus rules): proof · anchor is a known main-chain root · nf not in the nullifier set · one SpendAuthSig per Action + the binding signature. Then two state changes: the nullifier set gains nf_{old} , the tree gains cmx_{new} .

The chain gains a nullifier and a commitment per Action. Notes stay hidden, but action counts, timing, fees, and net pool flows remain public.

Keys: who can see what

The key ladder: one secret, graded capabilities



The full viewing key sees incoming notes and *spent-status*: holding *nk* it computes each note's nullifier and matches the chain — full balance history, no spend authority.

Outgoing recovery requires the sender to have used the corresponding *ovk*.

One key, 2^{88} unlinkable addresses

$$\begin{aligned}d_j &:= \text{FF1-AES256}_{dk}(j), & \text{pk}_d &:= [\text{ivk}] g_d, \\g_d &:= \text{DiversifyHash}(d_j)\end{aligned}$$

- A keyed permutation of the index space, not a counter and not a hash:
- *unlinkable* — consecutive indices scatter over $\{0, 1\}^{88}$; consecutive diversifiers would reveal a shared owner;
- *bijective* — never collides, and the wallet inverts it to recover the index behind any of its addresses, storing no per-address state;
- *total* — every one of the 2^{88} diversifiers is valid (a specified fallback replaces a hash-to-identity result).
- Give each payer their own address; all funds land in one wallet; linking two addresses of one owner is a DDH problem.

How the recipient finds the money

Nothing on chain marks whose output is whose — *detection is successful decryption*.

$$\text{epk} := [\text{esk}] g_d, \quad [\text{ivk}] \text{epk} = [\text{ivk}][\text{esk}] g_d = [\text{esk}] \text{pk}_d = s \rightarrow K_{\text{enc}}$$

- A fresh one-shot Diffie–Hellman per note; the wallet trial-decrypts each relevant pool's outputs. Full ciphertexts carry an AEAD tag; compact prefixes instead rely on commitment and ephemeral-key checks.
- Before accepting, the wallet re-derives esk and checks $[\text{esk}]g_d = \text{epk}$, then recomputes the commitment against the on-chain cmx — forgery closes here, outside the SNARK.
- A second ciphertext C_{out} , sealed under a key derived from ovk, lets the *sender or an auditor* recover the same note when outgoing recovery was enabled.
- Light clients scan on 52-byte compact prefixes — lead byte, diversifier, value, note seed.

The privacy sliders

slider	artifact	grants
spend	sk	move the money
full view	fvk	incoming + spent-status; recoverable outgoing
incoming view	ivk	detect and read incoming only (processor)
address	(d, pk_d)	receive; 2^{88} unlinkable ones per key

- “Viewing” names a *capability*, not publicity: every viewing key is secret material, disclosed deliberately, to confer exactly that visibility. Only the address is public.
- Validation needs no viewing key. This hides ordinary shielded note contents under the cryptographic assumptions; coinbase outputs and public transaction metadata are exceptions.

The proving machine

What a zk-SNARK promises

- *Complete* — an honest prover who knows the witness always convinces the verifier.
- *Knowledge sound* — “knowledge” is meant literally: a satisfying witness can be *extracted* from an adversarial prover that succeeds above the knowledge-error bound, under the assumptions.
- *Zero-knowledge* — the verifier finishes convinced yet learns nothing beyond the bare truth of the claim.

“Soundness protects against false statements; knowledge soundness protects against provers who do not actually know what they claim.”

For Zcash: a prover knows the witness for each enabled, nonzero spend. The separate SpendAuthSig supplies spend authority; a delegated prover need not hold the spending key.

The trusted-setup problem

A powers-of- τ example of structured parameters (not the complete BCTV14 or Groth16 parameter set):

$$\text{pp} = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \hat{G}, [\tau]\hat{G}), \quad \tau \text{ discarded}$$

- Retaining the relevant setup trapdoors can undermine soundness: **forged proofs could counterfeit ZEC without detection.**
- Ceremony trapdoors alone do not decrypt Zcash notes. Soundness and zero knowledge are separate guarantees; this is not a claim that every pairing-based scheme remains private under a malicious setup.
- Mitigation: multi-party ceremonies — six parties for Sprout; Powers of Tau plus a per-circuit phase for Sapling. Each phase needs an honest contribution with its secret erased.
- The setup assumption: at least one participant contributed honest randomness and destroyed their secret. Correct circuits, sound proofs, and the remaining consensus checks are still necessary.

Three eras

	Sprout (2016)	Sapling (2018)	Orchard (2022–)
SNARK	BCTV14	Groth16	Halo 2
curve	BN254	BLS12-381 + Jubjub	Pallas + Vesta
trusted setup	per-circuit (6-party)	per-circuit (large MPC)	none
in-circuit hash	SHA-256	Pedersen	Sinsemilla
proof size	296 B	192 B	~5 kB (one-Action bundle)
verification	constant	constant (3 pairings)	shared linear MSM
curve cycle	no	no	yes (unused by Orchard)

- A flaw in BCTV14 discovered in 2018 would have allowed undetected counterfeiting; Sapling activation replaced the vulnerable Sprout proving system with Groth16. This did not automatically migrate funds.
- The vulnerability came from a *proof-system bug*, not from toxic waste — even a perfectly honest ceremony would not have prevented it.

Halo 2 intuition: one secret becomes four rows

Public claim: “I know x such that $x^3 + x + 5 = 35$.”

Peek at the prover's private work:

$$\begin{aligned}x &= 3, & x^2 &= x \cdot x, \\x^3 &= x^2 \cdot x, & s &= x^3 + x, \\& & 35 &= s + 5.\end{aligned}$$

row	a	b	c
$x \cdot x = x^2$	3	3	9
$x^2 \cdot x = x^3$	9	3	27
$x^3 + x = s$	27	3	30
$s + 5 = 35$	30	5	35

Each elementary operation becomes one gate row.

The values are private *advice*; the gate rules and 35 are public. Copy constraints force every repeated x , x^2 , x^3 , and s to agree — turning four local facts into one computation.

Halo 2 intuition: rows become one identity

Label the four rows by the fourth roots of unity in $\mathbb{F}_{97}$:

$$H = \{1, 22, 96, 75\}, \quad Z_H(X) = \prod_{h \in H} (X - h) = X^4 - 1.$$

Interpolate each column into a low-degree polynomial. Combining those polynomials with the fixed gate selectors gives $G(X)$, whose value at each row label is exactly that row's gate equation:

$$G(h) = 0 \text{ for every } h \in H \iff G(X) = t(X) Z_H(X).$$

A correct gate trace is a polynomial identity; a bad trace leaves a non-zero remainder.

The Orchard Action circuit is the same machine at scale: $2^{11} = 2048$ rows, ten advice columns, copy constraints, and a lookup table for Sinsemilla's 10-bit chunks.

Halo 2 intuition: one random point catches a lie

1. The prover commits to the advice and quotient polynomials.
2. The transcript supplies a random challenge z .
3. The prover opens the commitments at z ; the verifier checks only

$$G(z) \stackrel{?}{=} t(z) Z_H(z).$$

If one row lies, then $D(X) := G(X) - t(X)Z_H(X)$ is a fixed, non-zero, low-degree polynomial. It can vanish only at a few points:

$$\Pr_z[D(z) = 0] \leq \frac{\deg D}{|\mathbb{F}|}.$$

Changing only the first row's c to 10 gives remainder $24(1 + X + X^2 + X^3)$: three roots out of 97. This bounds one idealised polynomial test; the full proof also needs binding, transcript security, copy constraints, and public-input checks.

Halo 2: why there is no ceremony

Commit with Pedersen over publicly derived generators (hash-to-curve — no ceremony secret is generated):

$$\text{Commit}(p; r) := \langle \vec{a}, \vec{G} \rangle + [r] H$$

- Opening proofs by logarithmic halving: $O(\log d)$ group elements, assumption discrete log only.
- The price: one linear-time multi-scalar multiplication sits at the end of verification.
- The Halo trick (*accumulation*): that check is itself a commitment opening of a structured polynomial — so defer it into a running accumulator and decide once. Deployed Orchard uses ordinary randomised batch verification with a shared MSM, not recursive Halo accumulation.
- Causality, precisely: **transparency comes from the IPA's public-coin parameters; batching shares verification work.**
- Each Pasta curve's base field is the other's scalar field. This enables efficient alternating-curve recursion; Orchard does not use it.

“A Zcash full node checks committed polynomial identities, openings, signatures, and public state. Together, under the protocol’s security assumptions, these establish a valid transfer without exposing its private notes.”

Honest limits

What is still printed on the envelope

For a *fully shielded* transaction, an on-chain observer still sees:

- **Action counts** — visible separately for Orchard and Ironwood.
Cross-address-enabled bundles pad to a floor of 2 and then reveal $\max(2, \text{spends}, \text{outputs})$; the restricted post-NU6.3 Orchard builder instead needs separate spend and output actions;
- **fee** — derived from public value balances and transparent inputs/outputs.
ZIP-317's recommended fee reduces arbitrary fee fingerprints but still depends on the transaction's shape;
- **expiry height** — the ordinary builder default is target + 40; callers and specialised transaction policies may choose others;
- **anchor and timing** — which tree, when mined, when it hit the mempool.

A sealed envelope with the postmark, weight class, postage, and sell-by date printed on the outside — uniform *only if* every wallet uses the same defaults. The deployed anti-fingerprinting strategy is *conformity*.

The boundary is metered

- Amounts are hidden *inside* a pool; every crossing is a public signed field: the net flow v_{balance} (positive = value leaving the pool) is exact, per transaction.
- Pool migrations (Sprout retirement then, Orchard $\rightarrow$ Ironwood at NU6.3) pass through the same public turnstile: per-transaction amounts remain clear. ZIP-318 mitigates linkage with canonical denominations, transaction shapes, shared anchor buckets, and randomised delays; it does not hide a crossing amount.
- Paying a transparent-only recipient from shielded funds (ZIP-320) transits funds through an ephemeral transparent output; both that output and the subsequent transparent payment are publicly visible.
- The default change strategy can retain a zero-valued change output at exact balance, obscuring an exact-sum guess. Dust-to-fee and other policies are exceptions; this is not a consensus guarantee.

The server sees your requests

Light-wallet keys never leave the device — *requests do*. The deployed sync protocol leaks, to the server:

observable	reveals	mitigation status
lowest historic range	approximate birthday	no default padding at pin
range boundaries	scan state and progress	bucketing proposed
full-tx fetches	transactions of interest	ZIP-307 cover recommendations
submission channel	full tx pre-broadcast, peer IP	Tor capability, deployment unknown
transparent-address RPCs	addresses outright	jitter (ephemeral only)

- Without cover traffic, requested txids expose wallet interests; these may include sent transactions and transparent-graph dependencies.
- Structural, not accidental: a client that must ask for ranges reveals where it stopped; a client that must fetch what it detected reveals what it detected.

Frontier (design-stage, one line each)

- **Tachyon** — remove wallet scanning instead of patching its leaks: detection moves to the payment channel, absence-tracking to an oblivious service. The designers' own flagged risk: timing correlation of proof updates — “fatal to the entire approach” if unsolved.
- **Crosslink** — a finality layer under the PoW chain; about settlement assurance, not privacy.

Neither is deployed; neither has a complete consensus specification. The earlier slides distinguish protocol rules from wallet policy.

Takeaways

1. A shielded payment publishes commitments, nullifiers, ciphertexts, proofs, signatures, and metadata; ordinary note contents remain hidden.
2. “What do the ZKPs prove?” has a concrete, finite answer: ten checks — note integrity, gated membership, nullifier integrity, value integrity, key consistency, address binding, consensus gates.
3. Balance combines a Pedersen sum and one Schnorr signature whose secret key the builder can efficiently derive if the books balance, leaning on the circuit’s range checks to keep the sum honest.
4. Keys grade visibility: spend, view-all, view-incoming, receive — auditability is deliberate key handover.
5. Orchard removed the ceremony: public-coin commitments generate no toxic waste; batch verification shares the expensive group arithmetic.
6. The proofs hide the graph; the *systems around them* — fees, boundaries, sync servers — are where privacy actually leaks.

Backup

Backup: Sinsemilla

$$Q(D) := \text{GroupHash}^{\mathbb{G}}(\text{z.cash:SinsemillaQ}, D)$$

$$P[m] := \text{GroupHash}^{\mathbb{G}}(\text{z.cash:SinsemillaS}, \text{LE}_{32}(m))$$

$$\text{Sinsemilla}_D(M) := \text{Acc}_\ell, \quad \text{Acc}_0 := Q(D), \quad \text{Acc}_{i+1} := (\text{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \text{Acc}_i$$

$$\text{SinsemillaCommit}_r(D, M) := \text{Sinsemilla}_{D \parallel -\mathbf{M}}(M) + [r] H_D,$$

$$H_D := \text{GroupHash}^{\mathbb{G}}(D \parallel -\mathbf{r}, \varepsilon).$$

- Each 10-bit chunk costs one table lookup + two incomplete additions; a 510-bit message costs 51 rounds — the chunk-table structure maps directly onto Halo 2's lookup argument.
- Collision resistance (for fixed-length inputs) reduces to discrete log on Pallas, with generator derivation modelled as a random oracle; hiding comes from the uniform $[r]H_D$ term, unconditionally.

Backup: circuit and proof numbers

- One fixed PLONKish circuit, native field $\mathbb{F}_{p_{\text{Pallas}}}$; 2^{11} rows, ten advice columns, Sinsemilla lookup table.
- One aggregate proof per bundle: $2720 + 2272 \cdot n$ bytes for n Actions — 4992 bytes for one Action, including the $2 \log_2 2^{11} = 22$ IPA round elements.
- Action encoding 820 bytes; each signature 64 bytes.
- Proving uses FFTs and multi-scalar multiplications. For row count $d = 2^{11}$, an IPA has $O(\log d)$ rounds but a linear MSM remains; batching shares its fixed-base work. Bundle work also grows with n .
- The Merkle gadget stacks 32 layer-tagged hashes; Poseidon's only non-linearity is $x \mapsto x^5$.

Backup: field discipline

lives in	elements	role
$\mathbb{F}_{p_{\text{Vesta}}}$ (Pallas <i>scalar</i> field)	ask, rcm, rcv, r _{ivk} , α , esk, bsk	scalars
$\mathbb{F}_{p_{\text{Pallas}}}$ (Pallas <i>base</i> field)	nk, ρ , ψ , ivk, cmx, nf	field elements
$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ (Pallas points)	ak, pk _d , g _d , epk, cm, cv, rk	group elements
raw 256-bit strings	sk, dk, ovk, rseed	keys/seeds

The circuit's native field is $\mathbb{F}_{p_{\text{Pallas}}}$; the containment $p_{\text{Pallas}} < p_{\text{Vesta}}$ is what lets base-field elements act as scalars where needed.

Backup: the note plaintext and rseed

$$\text{np} = (\text{lead}, d, v, \text{rseed}, \text{memo})$$

Lead byte: 0x02 for Orchard, 0x03 for Ironwood. lead byte + 11-byte diversifier + 64-bit value + 32-byte seed + 512-byte memo = the 564-byte plaintext; compact prefix = first 52 bytes.

$$\text{esk} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(0x04 \parallel \rho))$$

$$\psi := \text{ToBase}(\text{PRFexpand}_{\text{rseed}}(0x09 \parallel \rho))$$

$$\rho := \text{nf}_{\text{old}} \text{ (of the note spent in the same Action)}$$

- The seed and paired nullifier derive note randomness. The recipient also needs the transmitted diversifier and value, and its viewing key.

Backup: the ZIP-244 v5 shielded sighash

$\text{sighash} := \text{BLAKE2b-256}(\text{ZcashTxHash_} \parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{trans}} \parallel d_{\text{Sap}} \parallel d_{\text{Orch}})$

- The 4-byte consensus branch ID keys the personalisation: a signature is bound to that consensus branch ID; competing chains sharing an ID do not gain replay protection from this field alone.
- The Orchard branch digests every Action's nullifier, commitment, value commitment, rk, and ciphertexts, plus flags, balance, anchor.
- Altering these signed fields invalidates the signatures, barring a forgery or hash collision. For a purely shielded v5 transaction the sighash coincides with the txid. ZIP-229 extends the digest to v6 with a separate Ironwood branch; authorising data is excluded from the txid.

Backup: a pedagogical IPA fold

Symmetric, unmasked fold; deployed Halo 2 also uses a masking commitment and a different folding convention.

$$\mathcal{R}_{\text{IPA}} := \left\{ ((P, x, v); (\vec{a}, r)) : P = \langle \vec{a}, \vec{G} \rangle + [r]H, \quad v = \langle \vec{a}, \vec{b}(x) \rangle, \quad \deg p \leq d-1 \right\}$$

$k = \log_2 d$ halving rounds; final check

$$P + [v] U + \sum_{j=1}^k ([u_j^2] L_j + [u_j^{-2}] R_j) \stackrel{?}{=} [a] G^{(0)} + [r^*] H + [a \cdot b^{(0)}] U$$

- The verifier evaluates the structured polynomial $g(X; u_1, \dots, u_k)$ in $O(\log d)$ — but computing $G^{(0)} = \text{Commit}(g; 0)$ is an $O(d)$ MSM. That asymmetry is the seed of accumulation: $G^{(0)}$ is itself a *commitment* to a structured polynomial, so the deferred check is itself an opening claim that folds into a running accumulator, decided once.

Backup: NU6.3 pool sealing

- Two pools, one protocol: the Orchard *protocol* (curves, hashes, circuit, keys) is shared; the Orchard and Ironwood *pools* each have their own tree, anchor, and nullifier set. Addresses work in both.
- From NU6.3 the legacy Orchard pool is sealed: coinbase must carry an empty Orchard component; every Orchard-pool Action must disable cross-address transfers (A10); and per transaction $v_{\text{balance}}^{\text{Orchard}} \geq 0$ — value exits, never enters.
- The cross-address rule is enforced via the verifying key consensus selects by branch (v5 transactions cannot bypass it); the coinbase and turnstile rules are checked outside the proof (the coinbase restriction applies specifically to coinbase transactions).

Backup: quantum footnote

- A future discrete-log-breaking adversary who *knows an address* (d, pk_d) computes $ivk = \log_{g_d} pk_d$ and replays the trial-decryption loop over the recorded chain: full receiving history for every diversified address sharing that incoming viewing key, past and future, for ciphertexts encrypted by this scheme.
- Recovering ivk alone does not reveal the spending key. The specific attack above requires a known address; its absence is not a proof of unconditional chain privacy. Ciphertexts and finite secrets remain computationally protected, and Shor also breaks proof soundness and signatures.

Sources and scope

This talk summarises the *Ironwood*, *Halo 2*, *Wallet*, and *Sync* Guides. The specification and ZIPs are authoritative.

- Zcash Protocol Specification: Action statement, Sinsemilla, note encryption, RedDSA.
- ZIP-229 and ZIP-258: v6, Ironwood, pool rules; ZIP-244: v5 digests.
- Orchard Book, “Nullifiers”: separate privacy and uniqueness assumptions.
- ZIP-307, ZIP-317, and ZIP-318: sync, fees, migration policy.
- Implementation snapshots follow the guides: Orchard 0.15.0; librustzcash e30517e4; Halo 2 261faacc. Wallet defaults describe these snapshots, not every wallet.

The four-row example isolates polynomial constraints; it is not a complete Halo 2 transcript or an overall soundness estimate.